Inferential statistics I

Marcin Włodarczak

Preliminaries

We start by loading `tidyverse`:

```
library('tidyverse')
theme_set(theme_bw())
```

We will also want to read in the file `08-inferential1-utils.R` containing additional functions for this class. These functions hide some of the complexity of R so that you can concentrate on the statistical concepts rather than on getting the code to run. If the following line produces an error, select `Session -> Set Working Directory -> To Source File Location` in RStudio.

```
source('08-inferential1-utils.R')
```

Finally, we will initialise the random number generator to make sure we are all getting the same results:

```
set.seed(500)
```

Sample mean

Let's draw a sample of 100 elements from a normal distribution with $\mu = 20$ and $\sigma = 5$.

```
pop_mean <- 20
pop_sd <- 5
samp <- rnorm(100, mean = pop_mean, sd = pop_sd)
```

Check its mean and standard deviation:

```
mean(samp)
```

```
## [1] 19.64609
```

```
sd(samp)
```

```
## [1] 5.311299
```

Since the samples are drawn randomly, each time we run the code above we will get a somewhat different value.

Sampling distribution of the mean

Using the population parameters from the previous section, we will now generate the sampling distribution of the mean and plot it. We achieve this by using `replicate` to draw 100,000 (remember that `1e5` is a shorthand for 1×10^5) samples of 100 elements and calculate the mean of each one.

```
n <- 100 # Sample size
samp_distr <- replicate(1e5, mean(rnorm(n, pop_mean, pop_sd)))
```

The result is thus a vector of 100,000 means:

```
length(samp_distr)
```

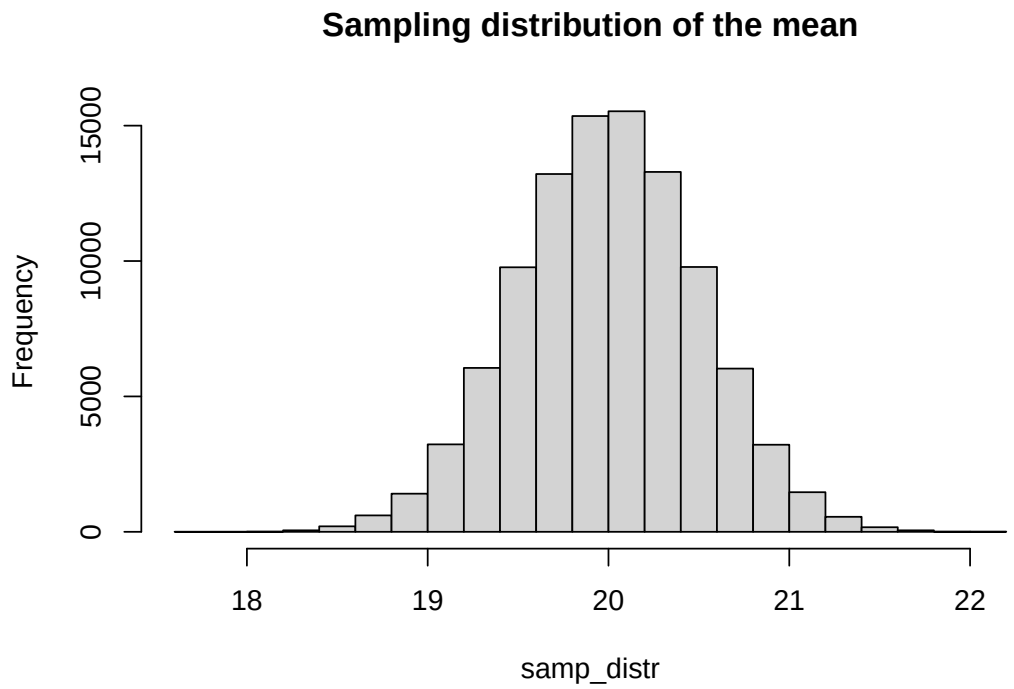
```
## [1] 100000
```

```
head(samp_distr)
```

```
## [1] 19.53162 19.43281 20.30485 19.94559 19.95223 19.95982
```

We can visualise their distribution using a histogram:

```
hist(samp_distr, main = 'Sampling distribution of the mean')
```



Notice that the mean of this distribution is very close to the population mean:

```
mean(samp_distr)
```

```
## [1] 19.99959
```

How about standard deviation?

```
sd(samp_distr)
```

```
## [1] 0.5005429
```

Note that the value above is indeed very close to standard error:

```
pop_sd / sqrt(100)
```

```
## [1] 0.5
```

Finally, we will demonstrate that for the sampling distribution of the mean, 95% of data lies in the interval $\mu \pm 1.96 \times \sigma_{\bar{x}}$:

```
se <- pop_sd / sqrt(100)
ci95_lo <- pop_mean - 1.96 * se
ci95_hi <- pop_mean + 1.96 * se
sum(samp_distr > ci95_lo & samp_distr < ci95_hi) / length(samp_distr)
```

```
## [1] 0.95023
```

Exercise

Experiment with different values of σ and different sample sizes to see how they impact standard error. For instance if you increase sample size by a factor of four, how much will standard error change?

Testing a single mean

Let's draw a sample of 100 data points from our population and check whether it did in fact come from this population (H_0):

```
samp <- rnorm(100, pop_mean, pop_sd)
```

The sample mean is:

```
mean(samp)
```

```
## [1] 20.90954
```

It is thus 0.91 away from the population mean. Let's divide this by standard error:

```
se <- pop_sd / sqrt(100)
z <- (mean(samp) - pop_mean) / se
```

In our case, the value of our test statistic is 1.819 and falls inside the 95% non-rejection region (-1.96, 1.96) so we cannot reject H_0.

Exercise

Repeat the steps above but this time simulate a situation where H_0 is false. For instance draw a sample from a population with $\mu = 10, \sigma = 5$ and check whether it came from a population with $\mu = 20, \sigma = 5$.

Confidence intervals

As we know, the 95% CI is equal to $\bar{x} \pm 1.96 \times \sigma_{\bar{x}}$:

```
mean(samp) + c(-1.96, 1.96) * se
```

```
## [1] 19.92954 21.88954
```

Of course, because this interval is centered around the sample mean, it will change from sample to sample.

Exercise

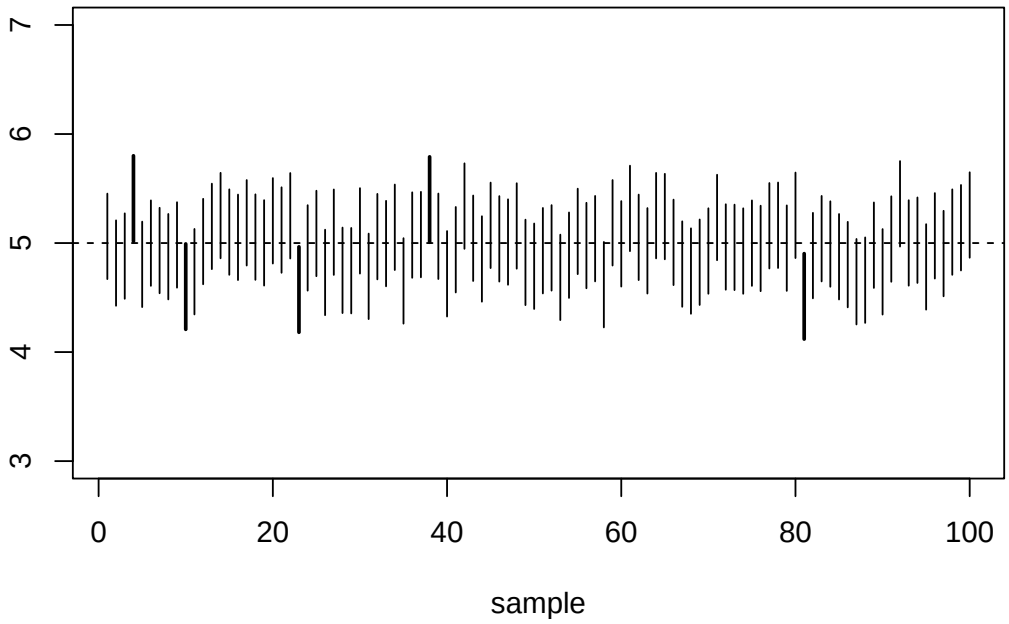
Run the code below a couple of times and see how the CI varies between samples. Can you see a CI which does no include the true population mean?

```
samp <- rnorm(100, pop_mean, pop_sd)
mean(samp) + c(-1.96, 1.96) * se
```

```
## [1] 19.3549 21.3149
```

As we saw last time, the idea behind the 95% confidence interval is that on *repeated sampling* it includes the population mean 95% of the time:

```
plot_ci_resamp(pop_mean = 5, pop_sd = 2, ci = 0.95)
```



Exercise

Run the code above a couple of times and observe how the CIs dance around the mean. Note that the number of CIs which do not include the population mean will not always be 5 due to random sampling.

Next, change the value of `ci` to 0.9 or 0.99 (in order to get 90% and 99% CIs) and see what happens. Do the CIs get wider or narrower?

One-sample t-test

Now let's assume we do not know the standard deviation of the population the sample came from (which is generally the case). In that scenario, we need to estimate it from sample standard deviation, which equals:

```
sd(samp)
```

```
## [1] 4.708178
```

We plug this value into the calculation of the test statistic

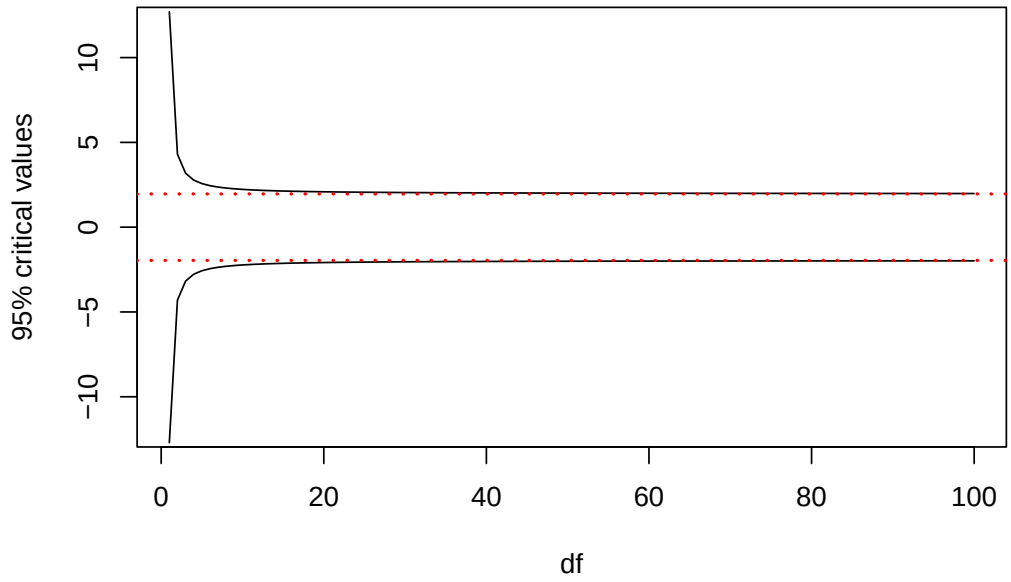
```
se_est <- sd(samp) / sqrt(length(samp))
t_val <- (mean(samp) - pop_mean) / se_est
```

In this case t equals 0.711. The number of degrees of freedom is 100 - 1 = 99. Let's get the critical values (you do not really need to try to remember how to do this in R):

```
crit95 <- qt(c(0.025, 0.975), df = length(samp) - 1)
```

Incidentally, the critical values we calculated are very similar to the critical values for the normal distribution. In fact, with increasing number of degrees of freedom, critical values for the two distributions converge

```
plot_t_crit(0.95)
```



You can get the associated p-value like this (you absolutely do not need to remember how to do it, either):

```
2 * pt(-abs(t_val), df = length(samp) - 1)
```

```
## [1] 0.4785597
```

In reality we would not calculate all this by hand. Instead we would use the `t.test` function to do all the work for us:

```
t.test(samp, mu = 20)
```

```
##
##  One Sample t-test
##
## data:  samp
## t = 0.71132, df = 99, p-value = 0.4786
## alternative hypothesis: true mean is not equal to 20
## 95 percent confidence interval:
##  19.40070 21.26911
## sample estimates:
## mean of x
##   20.3349
```

As you can see, R also calculates the 95% confidence interval for us. Of course we could also obtain it ourselves (remember that the confidence interval $\bar{x} \pm SE_{\bar{x}} \times t_{crit}$):

```
mean(samp) + se_est * crit95
```

```
## [1] 19.40070 21.26911
```

Note that in this case the 95% confidence interval includes the population mean.

Exercise

As an exercise, repeat the procedure in a scenario when H_0 is false. Does the 95% confidence interval include the population mean in that case?

```
# Write your solution here.
```

Two-sample t-test

We can also use the t-test to test differences between two samples (two-sample t-test). The logic in this case is very similar – assuming that the two samples come from populations with equal means (H_0), what is the likelihood of observing samples whose means differ at least as much as in the two samples in question. For a more in-depth discussion of the two-sample t-test, see the additional material below.

We can carry out a two-sample t-test with the `t.test` function if we supply it with two vectors:

```
x1 <- rnorm(50, 0, 1)
x2 <- rnorm(50, 4, 1)
t.test(x1, x2)
```

```
##
##  Welch Two Sample t-test
##
## data:  x1 and x2
## t = -19.847, df = 97.701, p-value < 2.2e-16
## alternative hypothesis: true difference in means is not equal to 0
## 95 percent confidence interval:
##  -4.565231 -3.735244
## sample estimates:
##    mean of x    mean of y
## -0.002943993  4.147293264
```

Note that in the example above the 95% confidence interval does not include 0.

If we put the data in a date frame (or a tibble), we can also use the familiar formula syntax:

```
d <- tibble(x = c(x1, x2),
            group = rep(c('A', 'B'), each = 50))
t.test(x ~ group, d)
```

```
##
##  Welch Two Sample t-test
##
## data:  x by group
## t = -19.847, df = 97.701, p-value < 2.2e-16
## alternative hypothesis: true difference in means between group A and group B is not equal to 0
## 95 percent confidence interval:
##  -4.565231 -3.735244
## sample estimates:
## mean in group A mean in group B
##    -0.002943993     4.147293264
```

Exercise

Repeat the analysis above in a scenario when H_0 is true. Does the 95% confidence interval include 0 in this case?

```
# Write your solution here.
```

Exercise

Use the `ratings` data set from the `langaugeR` package and compare mean size ratings for `animal` and `plant` words.

```
library('languageR')
data(ratings)

ratings %>%
    select(Word, Class, meanSizeRating) %>%
    head()
```

```
##        Word  Class meanSizeRating
## 1    almond  plant         1.8912
## 2       ant animal         3.6275
## 3     apple  plant         2.4730
## 4   apricot  plant         1.7597
## 5 asparagus  plant         1.8660
## 6   avocado  plant         1.7737
```

```
                                        # Write your solution here.
```

Next, fit a corresponding regression model and inspect its summary. Look in particular at the `Coefficients` section. Can you see any parallels with the `t.test`?

```
# Write your solution here.
```

Bonus: Two-sample t-test explained

The two-sample t-test lets us to test whether two samples were drawn from populations with the same means (H_0). Let's start by drawing two samples from distributions with different means (i.e. where H_0 is false).

```
samp1 <- rnorm(150, 25, 5)
samp2 <- rnorm(100, 20, 5)
mean(samp1)
```

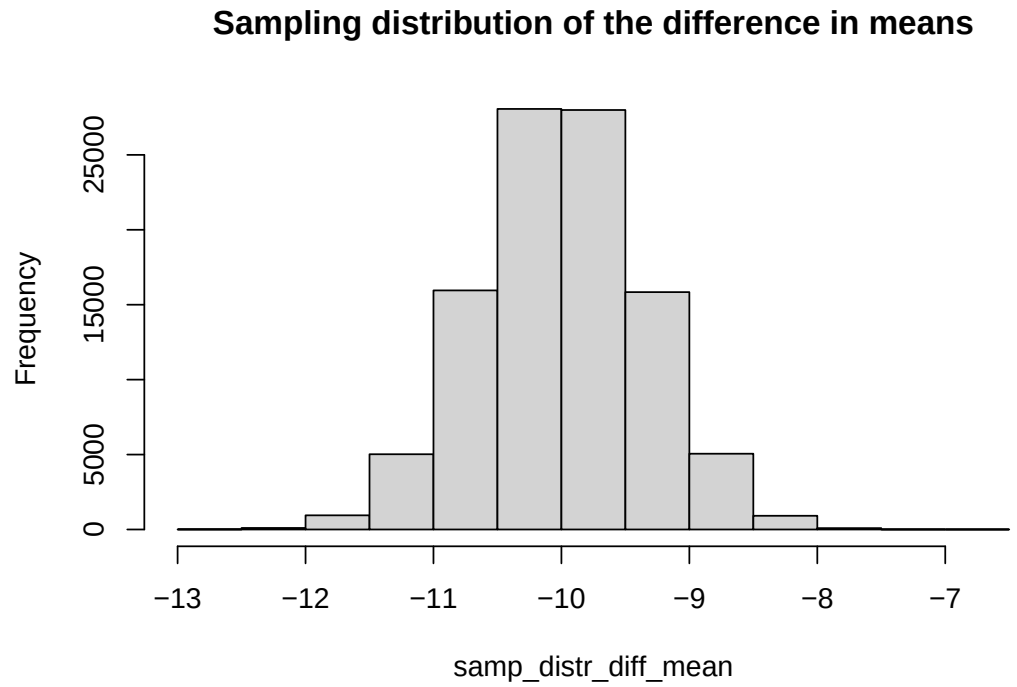
```
## [1] 24.5295
```

```
mean(samp2)
```

```
## [1] 20.92835
```

In this case, our sampling distribution is of the *difference of sample means*:

```
samp_distr_diff_mean <- replicate(
    1e5, mean(rnorm(150, 20, 5)) - mean(rnorm(100, 30, 5)))
hist(samp_distr_diff_mean, main = 'Sampling distribution of the difference in means')
```



Take a minute to make sure the mean of this distribution is equal to the difference between the population means. The standard standard deviation of this distribution is related to variances of both populations and is equal to:

$$\sqrt{\frac{\sigma_1^2}{n_1} + \frac{\sigma_2^2}{n_2}}$$

Since we do not know the value of σ, we need to estimate it from standard deviations of the samples. Assuming that the two two populations have the same σ^2 (which they do in this case), its unbiased estimate is given as:

$$s^2 = \frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n_1 + n_2 - 2}$$

The value above is called *pooled variance* since it pools (or combines) the standard deviations of the individual samples. Numerically, it is calculated as a weighted sum of the two variances. We can then use the estimated value and plug it into the formula for the standard error for the difference between two means:

$$SE_{\bar{x}_1 - \bar{x}_2} = \sqrt{\frac{s^2}{n_1} + \frac{s^2}{n_2}}$$

Are these two means different in this case? We start by calculating pooled variance and standard error:

```
n1 <- length(samp1)
n2 <- length(samp2)
var_pooled <- ((n1 - 1) * var(samp1) + (n2 - 1) * var(samp2)) / (n1 + n2 - 2)
st_err_diff <- sqrt(var_pooled / n1 + var_pooled / n2)
```

Again, note that the value of standard error calculated above (0.683) is a pretty good estimate of standard deviation of our simulated sampling distribution (0.647).

Now we are ready to calculate the t-value, the critical values and the p-value:

```
t_val <- (mean(samp1) - mean(samp2)) / st_err_diff
df <- n1 + n2 - 2
```

```r
crit95 <- qt(c(0.025, 0.975), df)
p_val <- 2 * pt(-abs(t_val), df, lower.tail = TRUE)
```

The value of the test statistic equals 5.273, the 95% critical values with 248 degrees of freedom correspond to (-1.97, 1.97), and p-value is 2.9219115×10^{-7}.

Of course, we can ask R to give us the answer:

```r
t.test(samp1, samp2, var.equal = TRUE)
```

```
##
##  Two Sample t-test
##
## data:  samp1 and samp2
## t = 5.273, df = 248, p-value = 2.922e-07
## alternative hypothesis: true difference in means is not equal to 0
## 95 percent confidence interval:
##  2.256043 4.946262
## sample estimates:
## mean of x mean of y
##  24.52950  20.92835
```

By default, when R carries out a t-test, it applies Welch's correction, which is necessary when variances of the two populations are not equal. Since in real life we almost never know population variances, this is what we want!

```r
t.test(samp1, samp2)
```

```
##
##  Welch Two Sample t-test
##
## data:  samp1 and samp2
## t = 5.1704, df = 197.75, p-value = 5.7e-07
## alternative hypothesis: true difference in means is not equal to 0
## 95 percent confidence interval:
##  2.227638 4.974668
## sample estimates:
## mean of x mean of y
##  24.52950  20.92835
```

As an exercise, draw two samples from the same population and repeat the procedure above.